



Introduction to Basic Data Structures

List Built-in Functions:

1. Constructor

Name	Details	Time Complexity
<code>list<type>myList;</code>	Construct a list with 0 elements.	O(1)
<code>list<type>myList(N);</code>	Construct a list with N elements and the value will be garbage.	O(N)
<code>list<type>myList(N,V);</code>	Construct a list with N elements and the value will be V.	O(N)
<code>list<type>myList(list2);</code>	Construct a list by copying another list list2.	O(N)
<code>list<type>myList(A,A+N);</code>	Construct a list by copying all elements from an array A of size N.	O(N)
<code>list<type>myList(v.begin(),v.end());</code>	Construct a list by copying all elements from a vector v.	O(N)

2. Capacity

Name	Details	Time Complexity
<code>myList.size()</code>	Returns the size of the list.	O(1)
<code>myList.max_size()</code>	Returns the maximum size that the list can hold.	O(1)

myList.clear()	Clears the list elements.	$O(N)$
myList.empty()	Return true/false if the list is empty or not.	$O(1)$
myList.resize()	Change the size of the list.	$O(K)$; where K is the difference between new size and current size.

3. Modifiers

Name	Details	Time Complexity
myList= or myList.assign(list2.begin(),list2.end())	Assign another list.	$O(N)$
myList.push_back()	Add an element to the tail.	$O(1)$
myList.push_front()	Add an element to the head.	$O(1)$
myList.pop_back()	Delete the tail.	$O(1)$
myList.pop_front()	Delete the head.	$O(1)$
myList.insert()	Insert elements at a specific position.	$O(N+K)$; where K is the number of elements to be inserted.
myList.erase()	Delete elements from a specific position.	$O(N+K)$; where K is the number of elements to be deleted.
replace(myList.begin(),myList.end(),value,replace_value)	Replace all the value with replace_value. Not under a list STL.	$O(N)$
find(myList.begin()	Find the value V. Not under	$O(N)$

<code>),myList.end(),V)</code>	a list STL.	
--------------------------------	-------------	--

4. Operations

Name	Details	Time Complexity
<code>myList.remove(V)</code>	Remove the value V from the list.	$O(N)$
<code>myList.sort()</code>	Sort the list in ascending order.	$O(N\log N)$
<code>myList.sort(greater<type>())</code>	Sort the list in descending order	$O(N\log N)$
<code>myList.unique()</code>	Deletes the duplicate values from the list. You must sort the list first.	$O(N)$, with sort $O(N\log N)$
<code>myList.reverse()</code>	Reverse the list.	$O(N)$

5. Element access

Name	Details	Time Complexity
<code>myList.back()</code>	Access the tail element.	$O(1)$
<code>myList.front()</code>	Access the head element.	$O(1)$
<code>next(myList.begin(),i)</code>	Access the ith element	$O(N)$

6. Iterators

Name	Details	Time Complexity
myList.begin()	Pointer to the first element.	O(1)
myList.end()	Pointer to the last element.	O(1)